

4501 Project Report

Game name: Frozen Fantasy

Team members

- Thomas Roche (ID 101260407)
- Artoria Zhou (ID 101089004)
- Ho Nam Leung (ID 101197127)

Game Summary

General Summary

Player controls an expanding group of units to gather resources, fight AI controlled enemies, and develop buildings with the eventual goal of slaying a dragon and gathering enough resources to build a building which restores fire to the frozen world (winning the game).

Obstacles

- Resources often guarded by enemies, requiring tactical approach to combat (send the right units at the right time etc.)
- World Event Manager can call in enemy raids etc. at night, meaning player has to strategize as the sky grows darker
- Development requires sacrifice, as the dangers of the world can't be overcome without sending units to die in battle.

Resources

- Wood, Stone, Firestone, and New Fire Shards are the player's main material resources (alongside the buildings and troops they possess)
- Resources gathered from deposits, done so by selecting units and directing them to the nodes (both via clicks)
- Buildings created by selecting from menu and placing in the world. Require resources, and non-pyre buildings must be near a pyre to be placed.

Gameplay Loop

- Player looks through map, decides which resources to target next
- Player assigns troops to attack any enemies guarding resources
- Battle ensues; player wins or loses
- On win, player gathers resources, and will use them to place new buildings or work towards the end objective.
- Either way, player recoups and deals with any world events which present themselves along the way.
- Repeat.

Changes In Development

Focus changed away from survival, leading us to remove food as a mechanic, since keeping all units fed was not a fun feature. This is especially true since we ended up with less wildlife than initially planned (we realized the 7 or so enemy units we had thought of were impractical).

The amount of units was reduced to accommodate for the difficulty of finding suitable animated models, creating appropriate stats, and implementing mechanical variety enough to justify the diversity. We still ended with 7 buildings and 6 dynamic units, so this is not a failing by any measure, just a reduction in a slightly overambitious scope.

While we had time to implement buffs and status effects had we wished, we elected not to, as it would require more units, more buildings, more time, and would bring the game back out of stable state. In other words, it was a risky proposition not worth pursuing.

A day/night cycle was added to accompany world events, accompanied by a storm of varying intensity etc.

A quest system was added to incentivise pursuing objectives such as gathering resources, creating buildings, killing enemies, etc. This grants resources and gradually unlocks player buildings as the game progresses, adding a very satisfying progression curve.

Technical Requirements

3D Rendering

Uses godot's inbuilt rendering capabilities. Light, shadows, etc. set up using appropriate values in our lighting system. All units textured and materialized to respond to light and render dynamically. Fairly standard.

Terrain

Uses custom heightmap terrain crafted and textured using a godot plugin. Uses NavMesh to enable navigation and has physical properties, meaning units get to use their physics processes with respect to it. Features a number of barriers, buildings (mostly enemy buildings to begin with) and collectible resources.

Units

Dynamic units with rules-based state machines (both enemy and player units). Have physics, pathfinding, a number of defined actions (specified in our scripts) etc.

Animation

Matches with unit state using simple script. Animations not custom (fully credited in liscence.txt), but adapted to work with the functions of our specific units.

Pathfinding

Dynamic units equipped with NavigationAgent nodes and pathfind on the terrain's NavMesh, which gets rebaked as buildings are created/destroyed. In most cases, dynamic units won't block other units in pathfinding because of how we've handled layers and collisions between units (they only collide in certain states).

Semi-autonomous action

Enemy units all are acting autonomously. Player units who have attacks will automatically go on the offensive if an enemy enters their vision area and they are not doing something else (in other words, player-directed actions won't be stopped). Further, when a unit being attacked dies, their attackers automatically find other enemies in range.

Enemy AI

Handled via state machines. Relatively simplistic, but enemies capable of sequential patrol, random patrol (roaming), attacking, etc. World events also allow enemies to coordinate and pursue objectives, like raids on the player's bases.

Game Mechanics

Quest system, world event system, etc. provide meaningful and interesting ways to engage with the world. Goals are clearly laid out, and player inputs are responded to accordingly. All resources serve a purpose, all building options provide different benefits to the player, and the gameplay loop is one that encourages a fun (if admittedly short) experience.

Player Feedback

The Quest system is the best player feedback system we have, but all player actions are directly reflected in the world. Unit deaths, building construction, the gathering of new resources, etc. are all reflected in the world or UI and make it obvious what kinds of outcomes the player's actions are leading to.

Extra Features

Quest System

- Tracks world state and gives appropriate unlocks and resources
- Good for player feedback; encourages good play

Day/Night System & World Events

- Player must strategise about what will happen when night falls
- Room for a good variety of events
- Looks good

This game is cool

- We're proud of the quality of the product.
- We have a dragon.

Class Hierarchy & Structure

Uses a combination of inheritance and component based. Good reliance on inherited subclasses to follow the behavioural model. Emphasis placed on unit customizability (buildings and units).

Main Classes summary:

- Main mostly handles terrain, building placement, and navmesh baking, with a few state functions, tracking of resources, and the functions to end the game
- WorldEventManager handles day/night cycle, weather, and world event processing. The specific events are in World Event subclasses
- GoalList manages quests and keeps track of world state as described elsewhere. It is the quest manager.
- UI manages UI. Clickable UI elements found here. Hover over buildings to display resource costs, etc.
- Camera also self explanatory; just handles camera controls.
- DynamicObject is the biggest class, as it handles general behaviour for units, their state machines and transitions, methods for processing states, etc.
- PlayerUnit/EnemyUnit handle unit control - player units can be clicked to select and direct them, enemy units can be set on paths from world events, etc.

The remainder of the classes are more minute and a bit more self explanatory. There is less interesting/important functionality there, and they mostly deal with the minute details of making sure the game runs. Of course, there is some interesting stuff, but listing and describing would be exhaustive.

CRC Cards

